Script — Harness Demo (15 min)

Text to say out loud or paraphrase. Marked with rough timings and with [LIVE: ...] where something needs to run in the terminal. No need to read it word for word — it's there to keep the thread, not to recite.

---

1. Opening — what Harness means to me (~1 min)

"When we were asked to bring our own interpretation of what a Harness is, that was the point: there's no single definition — it can be a skill, a runtime, a platform, a combination of tools. My interpretation is the most literal one: **a harness is the automated control system around a coding agent.** You give it specs *before* it writes code — that's feedforward — and you put automated gates on it that verify *after* what it produced — that's feedback. The core idea is that you don't review every line the agent writes: you design the conditions so that what it produces is correct by construction."

2. Why minimalist (~1 min)

"I deliberately decided not to go off on tangents. I could have built something with multiple repos, a Brownfield case, integrations — but the goal this week isn't to have a finished product, it's to validate the concept. So I picked the smallest case that would let me show the full cycle: specs → an agent that generates code → a gate that verifies it → a gate that catches a violation. A small 'Products' CRUD in .NET, Greenfield, so I could demonstrate and validate it end-to-end in the time I had. I'd rather show the complete mechanism on something small than show something big half-finished."

3. Walking through the project (~3 min) — while showing the folders

`specs/ -> the WHAT gates/ -> the automated control (the heart of the harness) scripts/ -> the feedback loop src/ -> empty. The app doesn't exist yet.`

> "This is Spec-Driven Development applied directly, not just in the background: the spec is the primary artifact, the code is its expression."

`specs/constitution.md`

> "These are the global rules, non-negotiable per feature: mandatory layered architecture — Api depends on Application, Application depends on Domain, Infrastructure is only touched by Application —, explicit prohibitions like controllers not being allowed to touch the DbContext directly, naming conventions, standard error handling. This isn't documentation someone can ignore — it's what the gates are going to enforce automatically."

`specs/products-spec.md`

> "The feature itself: the Product entity, the 5 REST endpoints, the business rules — price greater than zero, unique name, non-negative stock — and the edge cases. Notice it doesn't mention a single class, folder, or ORM. It's the WHAT, not the HOW — that's the SDD rule: the spec doesn't prescribe implementation."

`specs/verification.md`

> "Here's the traceability: every rule above becomes a checkable criterion with an ID — AC-01, AC-02, ARCH-01... — and every ID has a test with the same name in `/gates`. If a test fails, it traces directly back to the line in this document that originated it."

`gates/` — what a gate actually is

"This is the heart of the harness. A 'gate' is an automated check that decides whether code gets in or not — like a tollgate: it doesn't matter who wrote the code, human or agent, if it doesn't pass the gate, it doesn't get in. I have two kinds:

1. **Functional gates** (`Gates.Api.Tests`) — 13 xUnit tests, one per `AC-xx` criterion, that hit the already-running API over HTTP. They're *black-box*: they have zero compile-time reference to `/src`, so this project builds even when `/src` is empty.
2. **Architecture gates** (`Gates.Architecture.Tests`) — this is the interesting part. I use a library called NetArchTest, which doesn't read the source code — **it reads the already-compiled assembly**, the binary — and verifies real dependency relationships between layers. For example: 'no type in the Api assembly is allowed to depend on Microsoft.EntityFrameworkCore.' This is stronger than a style linter, because a linter looks at syntax; this looks at whether the compiler actually generated a reference to that assembly."

`scripts/verify.ps1`

"This is the feedback loop: it builds the gates, if `src/Api` already exists it boots the API, runs the 18 tests, and returns an exit code — zero if everything passed, non-zero if something failed. Red or green, no ambiguity. This is what orchestrates everything, and it's the same thing I'm going to ask the agent to run."

`src/`

"Empty on purpose, just a README saying not to implement anything yet. Key point: **everything in `/gates` already builds and runs even when `/src` is empty.** The harness doesn't need the app to exist in order to evaluate it — 'not implemented' is just as detectable a state as 'architecture violated'."

4. Live demo (~7 min)

Step A — show it red (1 min)

[LIVE: `.\scripts\verify.ps1`]

> "Gates build fine, but all 18 tests fail: the 13 functional ones because there's no API running, the 5 architecture ones because they can't find the assemblies under `/src`. This isn't an error in the harness — it's the harness working. It detects 'not implemented.'"

Step B — ask Copilot to implement it (~5 min)

[LIVE: paste the prompt into Copilot Chat, Agent mode]

> "I'm asking Copilot, in Agent mode, to implement `/src` following the constitution and the spec, without touching `/gates`, and to run `verify.ps1`, iterating until it goes green. While this runs, this is the feedback loop in action: the agent isn't asking me whether it's right, it's asking the gate."

[Wait for it to finish, run `verify.ps1` again if needed to confirm]

> "18 out of 18, green. The agent generated the full implementation — four projects respecting the layering — and the gate accepted it without me reviewing a single line by hand."

Step C — inject the violation and catch it (~2 min)

> "Now the part I most want to show you: what happens if someone — an agent or a person — violates the architecture?"

[LIVE: edit `ProductsController.cs`, add a `DbContext` field + `using Microsoft.EntityFrameworkCore;` (and the PackageReference in `Api.csproj` if needed for it to compile)]

[LIVE: `.\scripts\verify.ps1`]

> "ARCH01 fails, and not in a generic way — it points to the exact offending type: `Api.Controllers.ProductsController`. Everything else stays green. This is proof that the gate actually understands real architecture, not just syntax."

[Revert the change, run `verify.ps1` once more to close out green]

5. Closing (~1 min)

> "The core message: the spec was the feedforward, the gate was the feedback, and neither one required anyone to read the diff line by line. I deliberately picked something small and Greenfield, so I could show the complete cycle instead of a big piece half-done. This is my interpretation of Harness — probably different from each of yours, and that's exactly the point of this week."

In case they ask (short answers)

- **What if the agent edits the gates so they pass?** — That's a real gap in this minimal demo: the guardrail here is the prompt ("don't touch `/gates`") plus a human reviewing the diff. In a production version (like 8090), this gets solved with permissions: the agent literally has no write access to `/gates`.
- **Why NetArchTest instead of just a linter?** — A linter looks at syntax/style; NetArchTest inspects the compiled assembly and verifies real dependencies between layers.
- **Is this .NET-specific?** — The pattern (specs + gates + feedback loop) is stack-agnostic. In other languages the equivalent would be ArchUnit (Java), import-linter (Python), dependency-cruiser (JS/TS).
- **Why Greenfield instead of Brownfield?** — A deliberate scope choice for this week: validate the full mechanism in the time available. Adapting a Brownfield project to SDD is a logical next step, not something I skipped because it was technically hard.